FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Don Marco M. Sison
Section:	AN#42
Professor:	Ms. Crisola G. Tan
Date:	30/09/2026

Github:
https://github.com/dmsison-ph/Sison-DonMarco-CS0065-AN42/tree/main/TA4-Technical-Assessment

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

Part 1

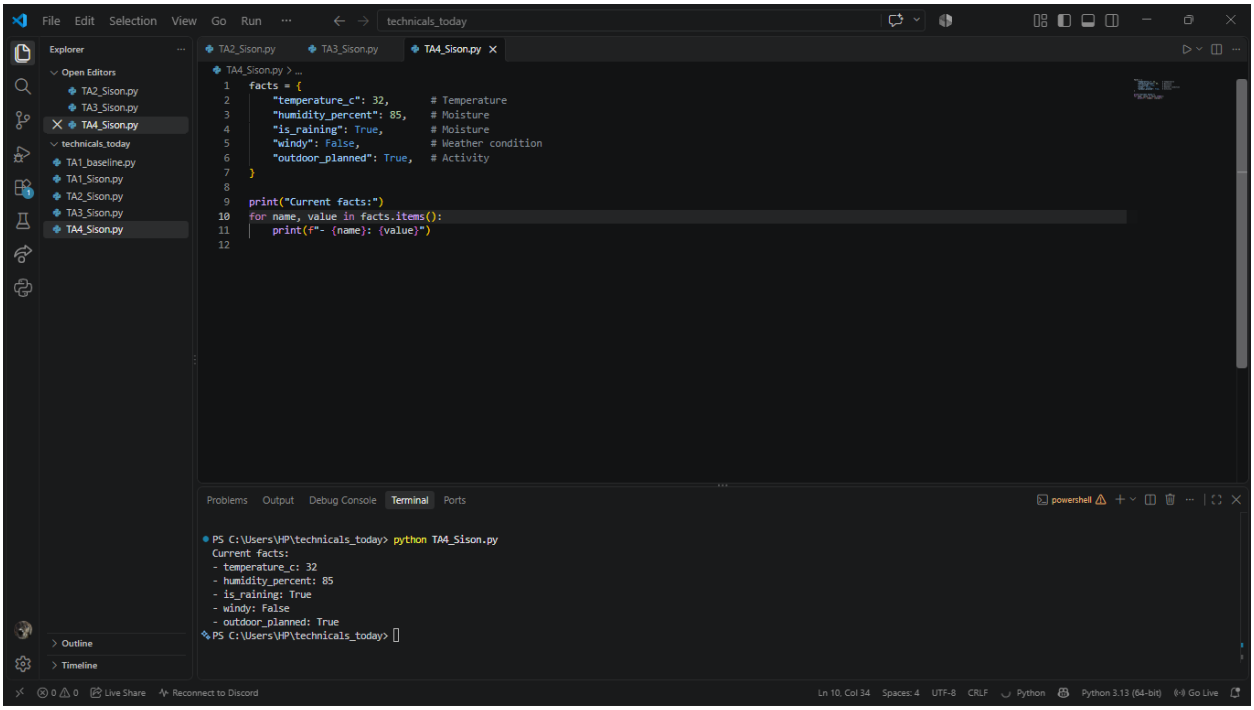
Code:

```
facts = {
   "temperature_c": 32,       # Temperature
   "humidity_percent": 85,    # Moisture
   "is_raining": True,        # Moisture
   "windy": False,            # Weather condition
```

```
    "outdoor_planned": True,   # Activity
}

print("Current facts:")
for name, value in facts.items():
    print(f"- {name}: {value}")
```

Screenshot:



Part 2

Code:

```
facts = {
    "temperature_c": 32,       # Temperature
    "humidity_percent": 85,    # Moisture
    "is_raining": True,        # Moisture
    "windy": False,            # Weather condition
    "outdoor_planned": True,   # Activity
}

print("Current facts:")
```

```
for name, value in facts.items():
    print(f"- {name}: {value}")

def rule_rain(facts):
    if facts["is_raining"] and facts["outdoor_planned"]:
        return "Bring an umbrella"

def rule_heat(facts):
    if facts["temperature_c"] >= 30 and facts["outdoor_planned"]:
        return "Bring drinking water"

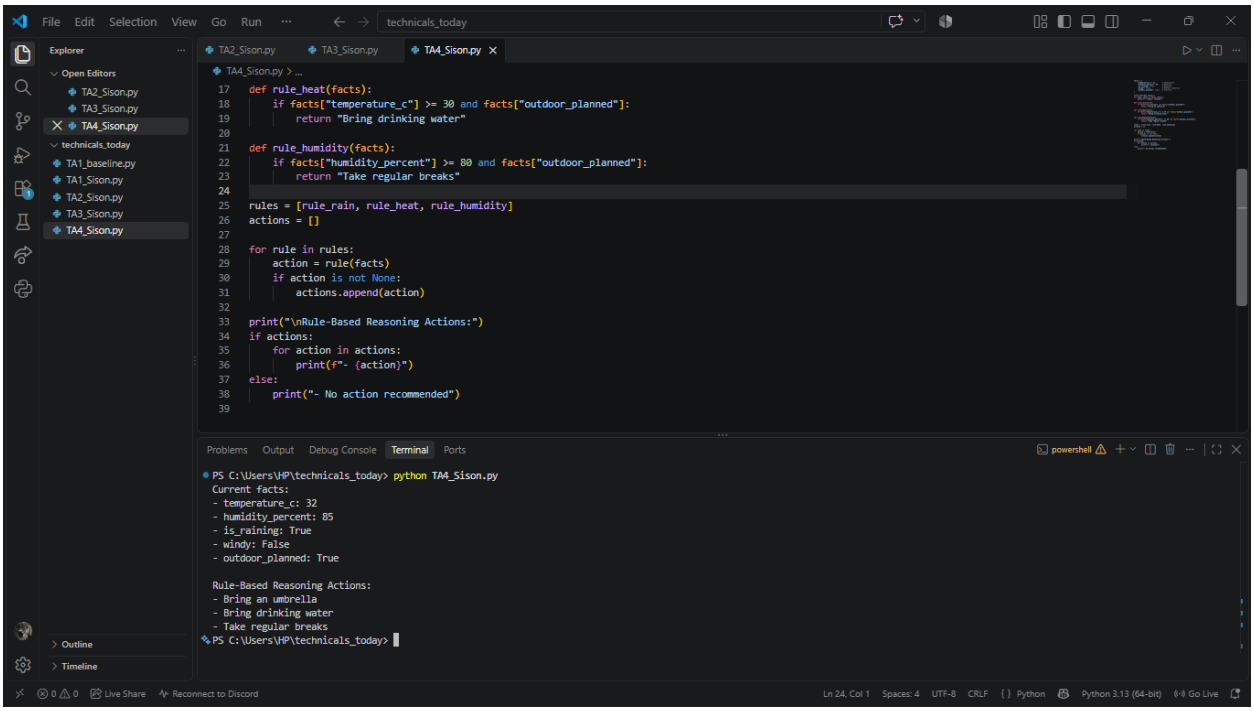
def rule_humidity(facts):
    if facts["humidity_percent"] >= 80 and facts["outdoor_planned"]:
        return "Take regular breaks"

rules = [rule_rain, rule_heat, rule_humidity]
actions = []

for rule in rules:
    action = rule(facts)
    if action is not None:
        actions.append(action)

print("\nRule-Based Reasoning Actions:")
if actions:
    for action in actions:
        print(f"- {action}")
else:
    print("- No action recommended")
```

Screenshot:



Part 3

Code:

```
facts = {
    "temperature_c": 32,       # Temperature
    "humidity_percent": 85,    # Moisture
    "is_raining": True,        # Moisture
    "windy": False,            # Weather condition
    "outdoor_planned": True,   # Activity
}

print("Current facts:")
for name, value in facts.items():
    print(f"- {name}: {value}")

def rule_rain(facts):
    if facts["is_raining"] and facts["outdoor_planned"]:
        return "Bring an umbrella"

def rule_heat(facts):
    if facts["temperature_c"] >= 30 and facts["outdoor_planned"]:
        return "Bring drinking water"
```

```
def rule_humidity(facts):
    if facts["humidity_percent"] >= 80 and facts["outdoor_planned"]:
        return "Take regular breaks"

rules = [rule_rain, rule_heat, rule_humidity]
actions = []

for rule in rules:
    action = rule(facts)
    if action is not None:
        actions.append(action)

print("\nRule-Based Reasoning Actions:")
if actions:
    for action in actions:
        print(f"- {action}")
else:
    print("- No action recommended")

case_base = [
    {
        "problem": {
            "temperature_c": 30,
            "humidity_percent": 80,
            "is_raining": True,
            "windy": False,
            "outdoor_planned": True,
        },
        "solution": "Bring an umbrella and water, and take breaks.",
    },
    {
        "problem": {
            "temperature_c": 24,
            "humidity_percent": 60,
            "is_raining": False,
            "windy": False,
            "outdoor_planned": True,
        },
        "solution": "Proceed with the outdoor activity.",
    },
    {
        "problem": {
            "temperature_c": 27,
            "humidity_percent": 75,
            "is_raining": True,
```

```
            "windy": True,
            "outdoor_planned": True,
        },
        "solution": "Consider postponing the outdoor activity.",
    },
]

print(f"\nPast cases available: {len(case_base)}")

def similarity(current, past):
    """Return a score from 0 to 1; higher means more similar."""
    temperature_score = max(
        0, 1 - abs(current["temperature_c"] - past["temperature_c"]) / 15
    )
    humidity_score = max(
        0, 1 - abs(current["humidity_percent"] - past["humidity_percent"]) / 40
    )

    rain_score = int(current["is_raining"] == past["is_raining"])
    wind_score = int(current["windy"] == past["windy"])
    activity_score = int(current["outdoor_planned"] == past["outdoor_planned"])

    return (
        temperature_score + humidity_score
        + rain_score + wind_score + activity_score
    ) / 5

print("\nCase-Based Reasoning Similarity Scores:")
for number, case in enumerate(case_base, start=1):
    score = similarity(facts, case["problem"])
    print(f"- Case {number}: {score:.3f}")

best_case = max(case_base, key=lambda case: similarity(facts, case["problem"]))
print("Most similar case:", case_base.index(best_case) + 1)

suggested_solution = best_case["solution"]
print("\nCBR suggested solution:", suggested_solution)

print("\nRevise the CBR solution:")
print("A. Keep the suggested solution")
print("B. Move the outdoor activity indoors because it is raining and humid")
print("C. Write my own solution")

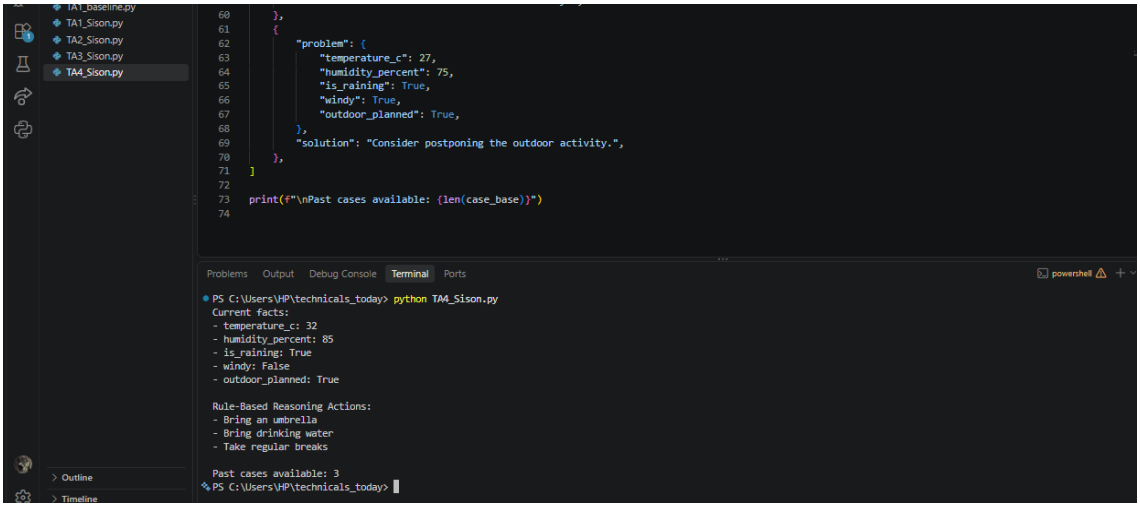
while True:
    choice = input("Choose A, B, or C: ").strip().upper()
```

```
    if choice == "A":
        final_solution = suggested_solution
        break
    elif choice == "B":
        final_solution = (
            "Move the outdoor activity indoors because it is raining and humid."
        )
        break
    elif choice == "C":
        final_solution = input("Enter your solution: ").strip()
        if final_solution:
            break
        print("Please enter a solution.")
    else:
        print("Please choose A, B, or C.")

print("CBR final solution:", final_solution)

case_base.append({
    "problem": facts.copy(),
    "solution": final_solution,
})
print(f"Cases after retaining today's case: {len(case_base)}")
```

Screenshots:



```
PS C:\Users\HP\technicals_today> python TA4_Sison.py
Current facts:
- temperature_c: 32
- humidity_percent: 85
- is_raining: True
- windy: False
- outdoor_planned: True

Rule-Based Reasoning Actions:
- Bring an umbrella
- Bring drinking water
- Take regular breaks

Past cases available: 3

Case-Based Reasoning Similarity Scores:
- Case 1: 0.948
- Case 2: 0.568
- Case 3: 0.683
Most similar case: 1

CBR suggested solution: Bring an umbrella and water, and take breaks.

Revise the CBR solution:
A. Keep the suggested solution
B. Move the outdoor activity indoors because it is raining and humid
C. Write my own solution
Choose A, B, or C: A
CBR final solution: Bring an umbrella and water, and take breaks.
Cases after retaining today's case: 4
```

```
PS C:\Users\HP\technicals_today> python TA4_Sison.py
Current facts:
- temperature_c: 32
- humidity_percent: 85
- is_raining: True
- windy: False
- outdoor_planned: True

Rule-Based Reasoning Actions:
- Bring an umbrella
- Bring drinking water
- Take regular breaks

Past cases available: 3

Case-Based Reasoning Similarity Scores:
- Case 1: 0.948
- Case 2: 0.568
- Case 3: 0.683
Most similar case: 1

CBR suggested solution: Bring an umbrella and water, and take breaks.

Revise the CBR solution:
A. Keep the suggested solution
B. Move the outdoor activity indoors because it is raining and humid
C. Write my own solution
Choose A, B, or C: B
CBR final solution: Move the outdoor activity indoors because it is raining and humid.
Cases after retaining today's case: 4
```

```
PS C:\Users\HP\technicals_today> Python TA4_Sison.py
Current facts:
- temperature_c: 32
- humidity_percent: 85
- is_raining: True
- windy: False
- outdoor_planned: True

Rule-Based Reasoning Actions:
- Bring an umbrella
- Bring drinking water
- Take regular breaks

Past cases available: 3

Case-Based Reasoning Similarity Scores:
- Case 1: 0.948
- Case 2: 0.568
- Case 3: 0.683
Most similar case: 1

CBR suggested solution: Bring an umbrella and water, and take breaks.

Revise the CBR solution:
A. Keep the suggested solution
B. Move the outdoor activity indoors because it is raining and humid
C. Write my own solution
Choose A, B, or C: C
Enter your solution: Reschedule the outdoor activity for a cooler, drier time. If it must proceed, bring an umbrella and water and take regular breaks.
CBR final solution: Reschedule the outdoor activity for a cooler, drier time. If it must proceed, bring an umbrella and water and take regular breaks.
Cases after retaining today's case: 4
PS C:\Users\HP\technicals_today>
```

Part 4

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

I found Rule-Based Reasoning an easier way to implement because of an if-then statement conditions for rain, heat, and humidity to check each rule in a loop. Case-Based Reasoning took more work because I had to create different steps for cases and implementing steps like retrieve, reuse, revise, and retain steps. This is to adapt and let the user know its solution. It could also improve as new cases are retained, although my current program stores them only until it closes. RBR is also useful when decisions follow clear rules like tax calculations while CBR is useful for comparing a new IT support problem with earlier solved cases.